

GEN-1 Pro 快速接入艾络迅云平台文档

目录

一、云端创建产品	3
1) 注册并登录开发者中心	3
2) 创建产品	4
二、功能定义（物模型）	8
三、激活码管理	9
四、设备端连云	10
1) 硬件准备与开发板组装	10
2) 安装驱动	11
3) 烧录 MCU 固件	11
4) 搭建开发环境（Thonny）	11
5) 编写连云脚本	12
6) 填写 PK/PS 鉴权信息并运行，设备上线	12
五、物模型数据上下行交互	14
1) 主动上报物模型数据（上行）	15
2) 响应平台读取请求（下行读取）	16
3) 主动上报告警事件（上行）	16
4) 接收平台服务下发（下行）	17
六、APP 使用	18
1) 为设备创建 SN 码（可用模组 IMEI 号或 自定义字符串）	18
2) 下载并登录“移联万物”	19
3) 扫码绑定设备	19

本文档说明从云端创建产品到设备真正上云、再到手机 APP 绑定的完整流程，帮助您端到端跑通整套方案。整体分为三个阶段：

阶段一 云端创建产品：在移远云开发者中心注册账号、创建产品、定义物模型（对应下文“创建产品”“功能定义”）。

阶段二 设备端连云：完成开发板组装、驱动安装、MCU 固件烧录与开发环境搭建后，编写并运行连云脚本，将产品鉴权信息（PK/PS/DK）填入代码，使设备接入平台并上报物模型数据（对应下文“连接平台的产品”）。

阶段三 APP 绑定：在“移联万物”APP 中扫码或配网添加设备，即可在手机上同步查看设备数据（对应下文“APP 使用”）。

一、云端创建产品

1) 注册并登录开发者中心

官方网址：<https://iot.quectelcn.com/login>

注册

[返回登录](#)

温馨提示

1、为了更好的为您提供IoT平台及软件解决方案，请根据您的需要选择合适的用户类型。

2、请输入您的邮箱地址，我们将发送注册链接至您的邮箱。填写的邮箱将会作为此账号唯一的申诉渠道，请谨慎填写。

*** 用户类型**

个人用户

10个

创建产品数量

5个

免费设备激活码



企业用户

300个

创建产品数量

10个

免费设备激活码

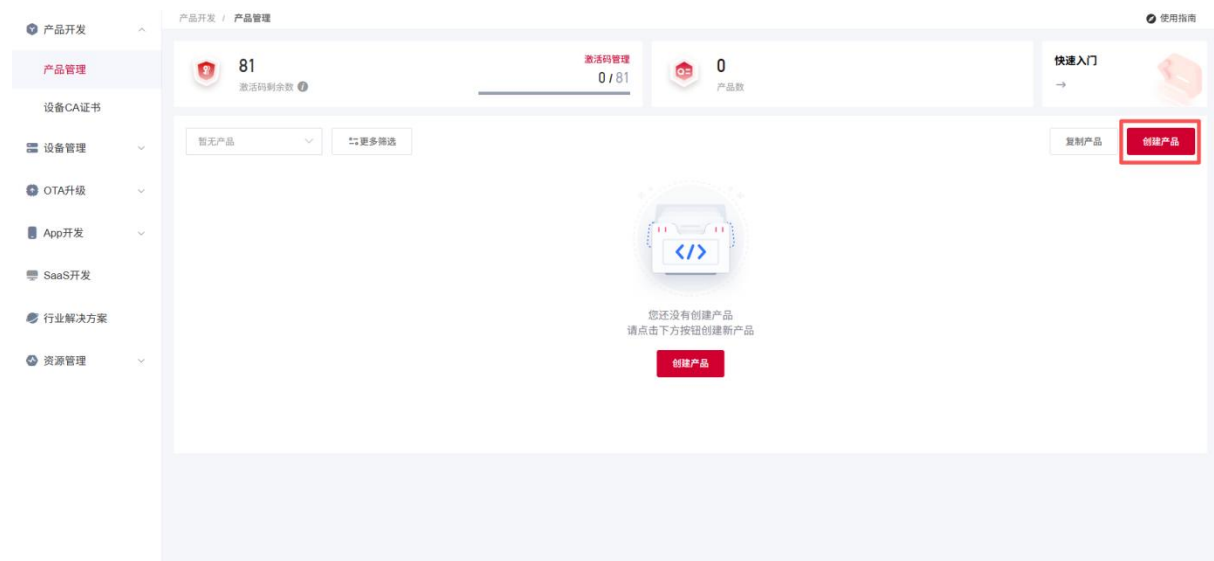


*** 注册邮箱**

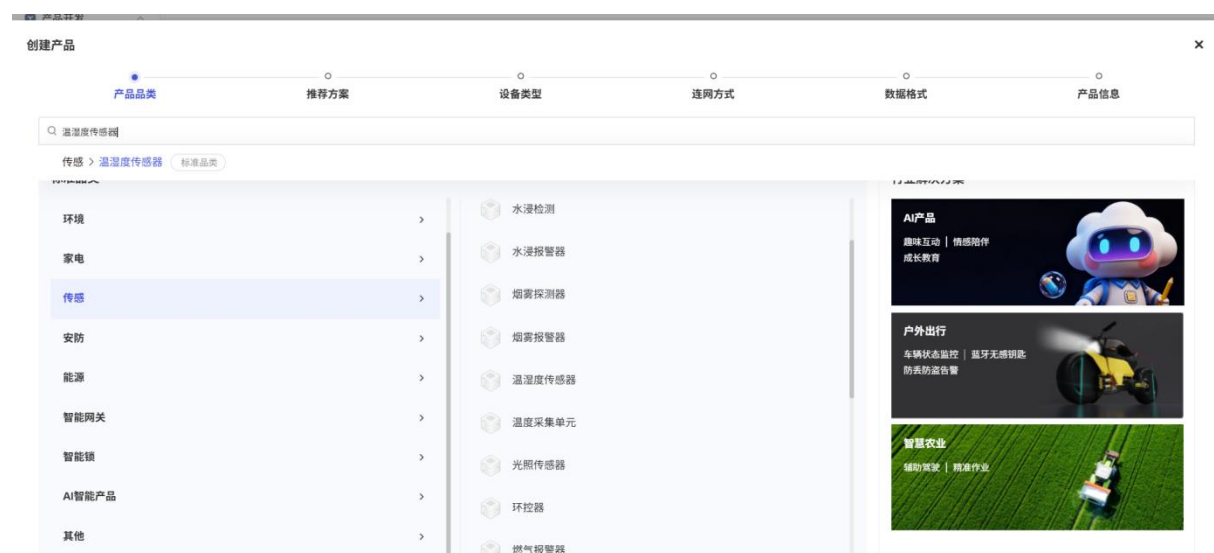
☐ 我已阅读并同意 [《服务协议》](#) | [《隐私政策》](#)

发送邮件

2) 创建产品



第一步：选择品类，通过品类搜索香薰机。



第二步：选择方案。您可选择系统中已预设的推荐方案完成快速创建，也可以通过自定义方案根据实际需要进行配置，此处我们以自定义方案进行配置。



第三步：选择产品的联网类型与通讯方式。此处我们选择**单品设备**类型、**2G/3G/4G/5G** 联网方式、数据格式使用**物模型**类型。

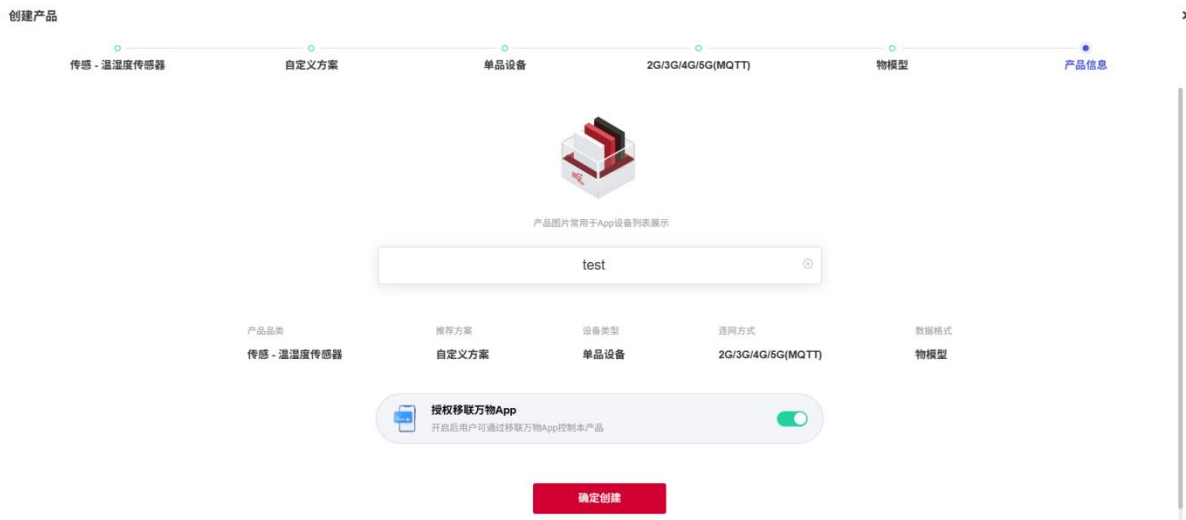




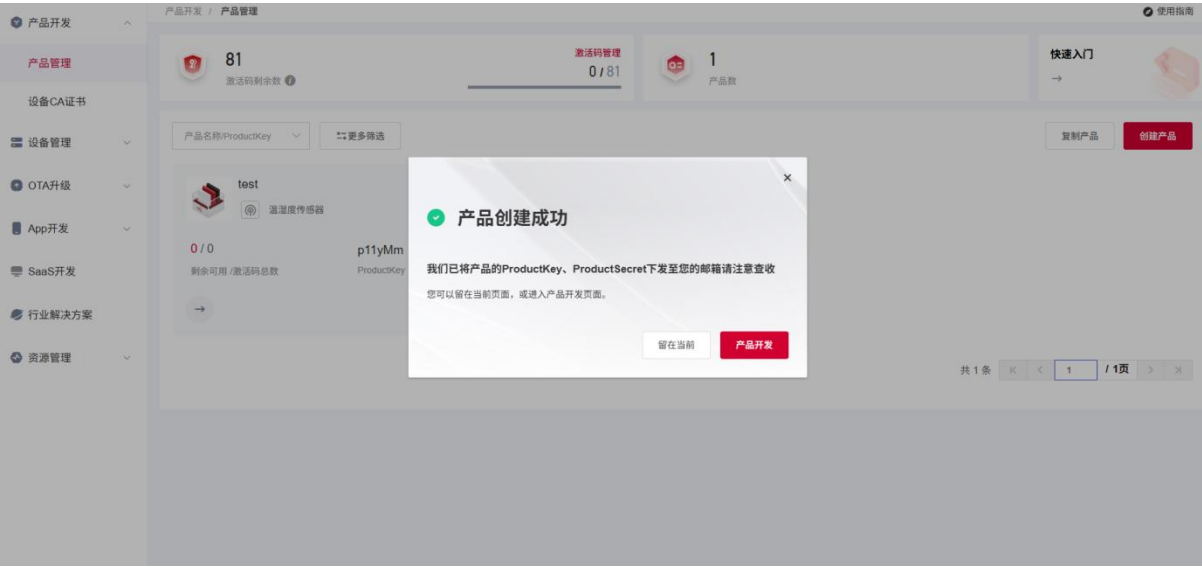
选择物模型数据格式。



第四步：最后上传产品图片、输入产品名称，提交并完成产品创建。

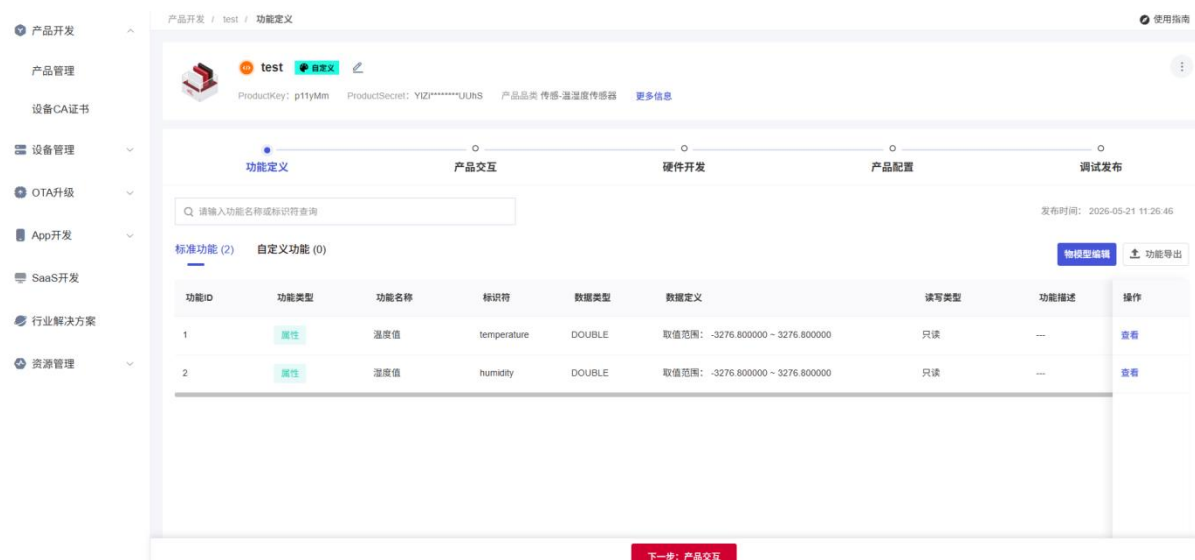


创建成功后平台将为该产品分配 ProductKey 与 ProductSecret，用于后续设备需连接平台时的鉴权，具体的信息会下发到账号的注册邮箱中。

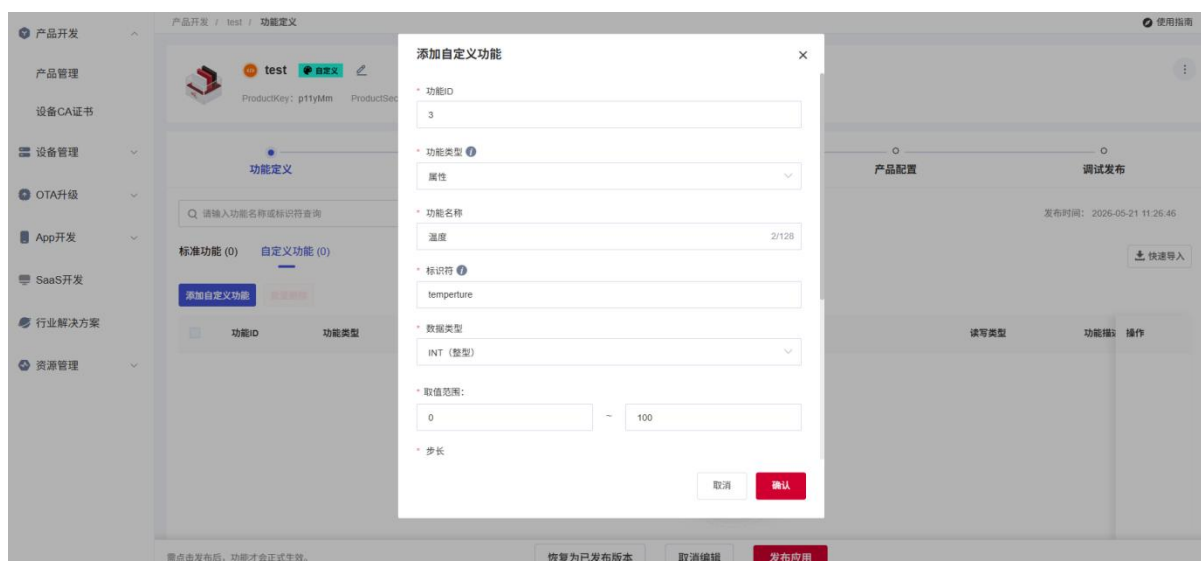


二、功能定义（物模型）

产品创建后可进入产品详情的功能定义页面进行物模型功能定义。



添加自定义功能



拟定物模型文档可点击跳转：<https://iot-cloud-docs.quectelcn.com/document/page-0503>

三、激活码管理

创建好产品后，第一步进行激活码分配。点击激活码管理后，点击分配激活码对需要分配的产品进行分配激活码。

资源管理 / 激活码管理

设备激活码 (又称"License")，设备激活码是设备连接开发者中心的唯一合法凭证。每台设备连接平台时会消费一个激活码。

激活码在购买时设置了有效期，当设备的激活码超过有效期时，将重新消费一个新的激活码，若激活码数量不足，则该设备会下线并重置为未激活状态，无法登录平台。

未分配激活码归属在账号下，需要分配给指定产品后可使用，产品激活码在下单时直接归属在指定产品下，不需要分配。也无法给其他产品使用。

注：此处仅展示账号所归属数据中心的激活码情况，仅允许在归属数据中心分配激活码。

激活码

630

总数

38

已使用

592

剩余可用

未分配激活码 ⓘ

18

查看 >

分配激活码

产品激活码列表

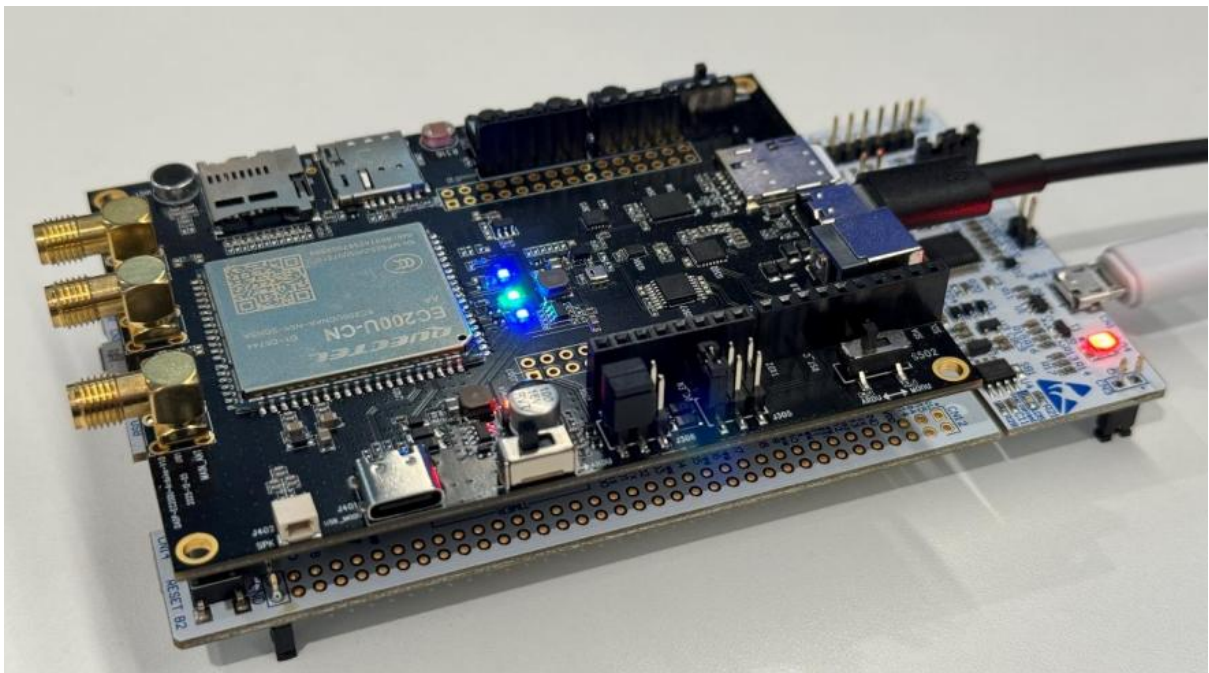
激活码分配记录

四、设备端连云

在把上面云端创建产品所得到的鉴权信息填入代码之前，需要先完成设备端的硬件与软件准备。本节以 UniKnect Gen1 Pro 开发套件（板载 EC200U 4G 模组 + NUCLEO-F413ZH 主控板）为例，介绍设备从硬件组装到运行连云脚本的完整步骤：硬件准备 → 安装驱动 → 烧录 MCU 固件 → 搭建开发环境 → 编写并运行连云脚本。

1) 硬件准备与开发板组装

将 UniKnect Gen1 Pro 扩展板与 NUCLEO-F413ZH 开发板通过 Arduino UNO 标准接口垂直堆叠插接组装为一体，两板间的 UART、GPIO、电源等关键信号将自动连通，无需额外飞线。组装时注意方向对齐（DC5V 电源输入端与 ST-LINK/V2-1 朝向一致），垂直下压完全插合。为保证板载 4G 模组供电充足，需接入 Type-C 供电线；主控板需连接 ST-LINK/V2-1 侧的 MicroUSB 数据线，用于烧录固件和调试。



2) 安装驱动

驱动用于让 PC 与开发板正常通信，请根据需要安装以下驱动：

① **4G 模组驱动（可选）**：如需通过 USB 对模组收发 AT 命令或抓取 log，安装 Quectel_Windows_USB_DriverU_V1.0.19.zip；安装后连接模组 USB 接口，设备管理器可枚举出对应端口即成功。若无需收发 AT/抓 log 可跳过。

② **调试器驱动（必需）**：NUCLEO-F413ZH 自带 ST-LINK/V2-1 调试器，用于烧录/调试 MCU，并虚拟出一路 UART 供 MicroPython 交互。下载安装 ST-LINK/V2-1 驱动：
<https://www.st.com.cn/zh/development-tools/stsw-link009.html>

③ **串口驱动（可选）**：板载 CP2105 USB 转串口芯片，用于捕获 4G 模组 debug log 及查看模组与 MCU 间的 AT 交互，需安装 CP210x Universal Windows Driver。若无需可跳过。

3) 烧录 MCU 固件

MCU 固件即 MicroPython 的软件运行环境。安装好 ST-LINK/V2-1 驱动后，按以下步骤烧录：

① 下载并安装烧录工具 STM32CubeProgrammer：
<https://www.st.com.cn/zh/development-tools/stm32cubeprog.html>

② 下载 MCU 固件 UniKnect_MicroPython_F413ZH：<https://github.com/quectel-develop/micropython/releases>

③ 打开 STM32CubeProgrammer，加载固件 → 连接设备 → 开始烧录 → 等待烧录完成。

4) 搭建开发环境（Thonny）

MicroPython 开发需要打通三个环节：运行环境（已烧录的 MicroPython 固件）、通信桥梁（ST-LINK 虚拟串口）与开发环境（IDE）。此处选用 Thonny 作为 IDE：

① 下载并解压 Thonny-4.1.7.zip（内置 Python 3.10，需 Windows 8.1/10/11 64 位），运行 thonny.exe。

② 点击【Tools】→【Options…】→【Interpreter】，在下拉列表选择 MicroPython (generic) 解释器以匹配 NUCLEO 开发板；在【Editor】中勾选前三个 Automatically 以开启代码自动补全；如需可在【General】设置中文、在【Theme & Font】调整主题字体（部分设置需重启生效）。

③ 连接 NUCLEO-F413ZH 的 ST-LINK/V2-1 接口至 PC，打开 Thonny，若 Shell (REPL) 窗口显示设备信息即表示设备已连接成功。

5) 编写连云脚本

在 Thonny 左侧 MicroPython device 文件区右键【New file...】，新建并命名为 main.py，编写连云脚本。设备端通过云 SDK 模块 Qth 接入平台：先用 setProductInfo 填入产品的 ProductKey/ProductSecret，用 setDK 填入设备 DK（可用模组 IMEI 号），调用 start() 完成注册与登录，连接成功后用 sendTsl 上报物模型数据。参考代码如下：

```
import Qth
import utime

# ===== 1. 填写云端创建产品后获得的鉴权信息 =====
PK = "your_ProductKey"      # 平台产品的 ProductKey
PS = "your_ProductSecret"   # 平台产品的 ProductSecret
DK = "your_DeviceKey"       # 设备 DK / SN（可用模组 IMEI 号）

# ===== 2. 事件回调 =====
def dev_event(event, errcode):
    print("dev event:", event, errcode)

def recv_tsl(data):
    print("recv tsl:", data)

# ===== 3. 初始化并连接云平台 =====
Qth.init()
Qth.setProductInfo(PK, PS)
Qth.setDK(DK)
Qth.setServer("mqtt://iot-south.quectelcn.com:1883")
Qth.setEventCb({
    "devEvent": dev_event,
    "recvTsl": recv_tsl,
})

if Qth.start():
    print("Qth start success")

# 等待连接成功
while not Qth.state():
    print("connecting ...")
    utime.sleep(1)
print("cloud connected")
```

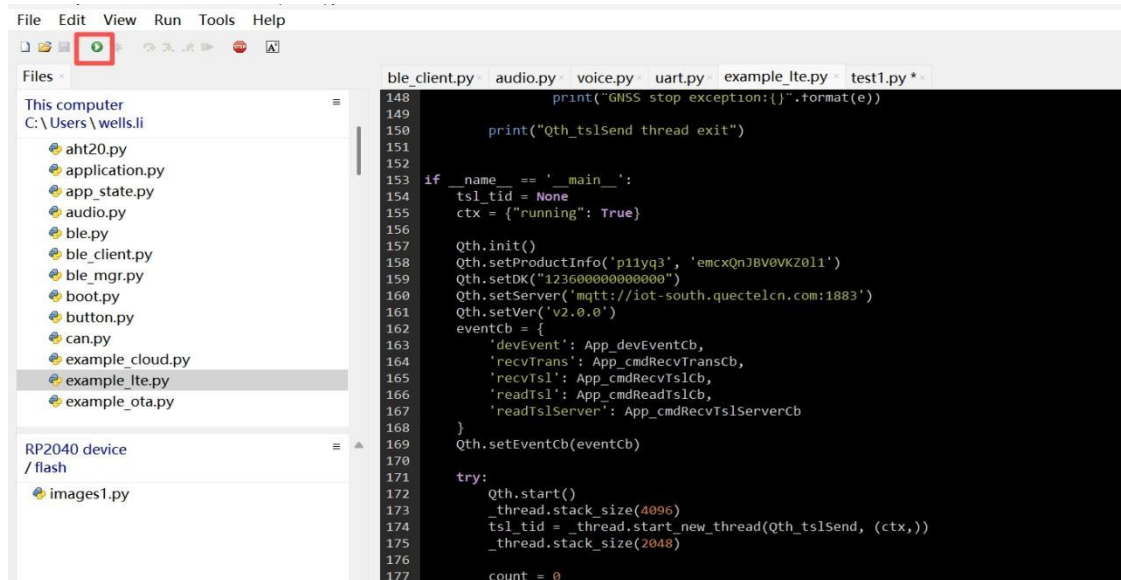
6) 填写 PK/PS 鉴权信息并运行，设备上线

将要获取到的 PK 和 PS 输入在代码中对应的位置，然后点击运行代码

```

158 Qth.init()
159 Qth.setProductInfo('p11yq3', 'emcxQnJBV0VKZ011')
160 Qth.setDK("1236000000000000")
161 Qth.setServer('mqtt://iot-south.quectelcn.com:1883')

```



连接成功后，可以在艾络迅云平台看到该台设备为在线状态。

设备列表 跨区域设备列表

33 已激活设备 1 禁用设备 32 离线设备

产品名称(ProductName) 设备Key(设备系统序列号) 请填入设备内容 全部状态 激活状态 禁用状态 搜索 新增

设备名称	DeviceKey	所属产品	设备类型	是否认证	激活状态	设备状态	最后上线时间	设备ID	ICCID	设备标识	应用状态	固件版本	软件版本	操作
net_2010	00000072123810	net	单设备	已认证	激活	在线	2020-05-22 11:25:28	00000072123810	8833012517303788	00000072123810	应用	E0802ZLLAR07AD...	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-02 10:30:59	—	00004F1062380816	—	应用	B080XOLAR01AD...	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-30 11:59:24	—	0000081310250000	00000009027800	应用	EC080MCLER08AD...	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-01 15:00:45	—	—	B0 0C 80 5F 45 55	应用	WFI01R13A0L_OCP...	[WCU1]	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-15 15:17:43	net2010	—	—	应用	B0802ZLLAR02AD...	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-14 15:37:17	—	0000081310250000	00402008144244	应用	EC080MCLER08AD...	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-14 10:47:41	00000071540405	0000081310250000	00000071540405	应用	E0802ZLLAR07AD...	[WCU1]	查看 详情 删除
			单设备	已认证	激活	离线	2020-05-10 15:03:27	00000072123810	00000072123810	00000072123810	应用	E0802ZLLAR07AD...	—	查看 详情 删除
			子设备	已认证	激活	离线	2020-04-20 20:50:02	—	—	—	应用	—	—	查看 详情 删除
			单设备	已认证	激活	离线	2020-04-20 18:41:30	—	00000072123810	00000072123810	应用	EC081ECN0908DA...	[Y112]	查看 详情 删除

五、物模型数据上下行交互

物模型是开发者中心针对物理设备在平台端建立的数据模型，用于描述产品功能、方便数据解析。设备成功接入平台后，通过构建物模型数据实现与平台的交互：上行使用 Qth.sendTsl() 上报数据，下行通过 readTsl / recvTsl 等回调接收平台下发的读取请求或服务指令，并用 Qth.ackTsl() 应答。

涉及接口：Qth.sendTsl()（发送物模型数据）、Qth.ackTsl()（应答物模型读取请求）。

功能 ID	功能类型	功能名称	数据类型	数值范围	引用关系
1	属性	开关	BOOL	true - 开启; false - 关闭;	
2	属性	电池电量	INT	0~100	
3	属性	精油剩余容量	FLOAT	0~300	
4	属性	工作模式	ENUM	1~HPC 2~CPL	
5	属性	设备编号	TEXT	0~20	
6	属性	时间	DATE	--	
7	属性	结构体	STRUCT	--	功能 ID: 8, 9
8(父 ID: 7)	参数	{ \$参数名称 }	BOOL	true - 开启; false - 关闭;	
9(父 ID: 7)	参数	{ \$参数名称 }	INT	0~100	
10	属性	整型数组	ARRAY	最大数量: 10, 元素 类型: INT	
11	属性	结构体数组	ARRAY		功能 ID:

功能 ID	功能类型	功能名称	数据类型	数值范围	引用关系
					12, 13
12(父 ID: 11)	参数	{ \$参数名称 }	BOOL	true - 开启; false - 关闭;	
13(父 ID: 11)	参数	{ \$参数名称 }	INT	0~100	

1) 主动上报物模型数据（上行）

设备读取传感器数据后，将各属性按“物模型 ID:值”组织为字典，通过 Qth.sendTsl() 上报到平台。示例：

```
# 创建一个空字典
my_dict = {}
# 布尔 ID1
my_dict.update({"1": True})
# 整型 ID2
my_dict.update({"2": 100})
# 浮点 ID3
my_dict.update({"3": 25.6})
# 枚举 ID4
my_dict.update({"4": 1})
# 字符串 ID5
my_dict.update({"5": "ZWS"})
# 时间(ms) ID6
my_dict.update({"6": 1659664758817})
# 结构体 ID7（含子参数 8、9）
my_dict.update({"7": {"8": True, "9": 100}})
# 整型数组 ID10
my_dict.update({"10": [10, 20, 30]})
# 结构体数组 ID11（子参数 12、13）
my_dict.update({"11": [{"12": True, "13": 100}, {"12": False, "13": 98}]})
# 发送数据，QoS 模式：1
Qth.sendTsl(1, my_dict)
```

2) 响应平台读取请求（下行读取）

在开发者中心“设备调试”中可点击“批量读取”一键读取物模型属性。模组收到读取指令后会自动触发 readTsl 回调，回调入参为请求的物模型 ID 列表 ids 与包 ID pkgId；按 ids 组织数据后用 Qth.ackTsl() 应答。示例：

```
def App_cmdReadTslCb(ids, pkgId):
    print('readTsl:', ids, pkgId)
    my_dict = {}
    for id in ids:
        if id == 1:
            my_dict.update({"1": True})
        elif id == 2:
            my_dict.update({"2": 100})
        elif id == 3:
            my_dict.update({"3": 25.6})
        elif id == 4:
            my_dict.update({"4": 1})
        elif id == 5:
            my_dict.update({"5": "ZWS"})
        elif id == 6:
            my_dict.update({"6": 1659664758817})
        elif id == 7:
            my_dict.update({"7": {"8": True, "9": 100}})
        elif id == 10:
            my_dict.update({"10": [10, 20, 30]})
        elif id == 11:
            my_dict.update({"11": [{"12": True, "13": 100}, {"12": False, "13": 98}]})
    # 应答读取请求, QoS 模式: 0
    Qth.ackTsl(0, my_dict, pkgId)
```

该回调需在 setEventCb 中通过 "readTsl" 字段注册。

3) 主动上报告警事件（上行）

事件型物模型（如 ID14 剩余精油不足）可携带输出参数（ID15 电池电量、ID16 精油剩余容量）组成结构体一并上报：

```
my_dict = {}
# 事件 ID14, 携带输出参数 15、16
my_dict.update({"14": {"15": True, "16": 20.5}})
# 发送物模型事件数据至平台
Qth.sendTsl(1, my_dict)
```


4) 接收平台服务下发（下行）

在开发者中心“设备调试→服务调用”中选择物模型服务并发送指令（如 ID17 服务，输入参数 ID18 模式）。模组收到后自动触发 `recvTslServer` 回调，解析输入参数进行处理：

```
def App_cmdRecvTslServerCb(serverId, value, pkgId):  
    print('recvTslServer:', serverId, value, pkgId)  
    if serverId == 17:  
        num = value["18"]  
        print('num:', num)
```

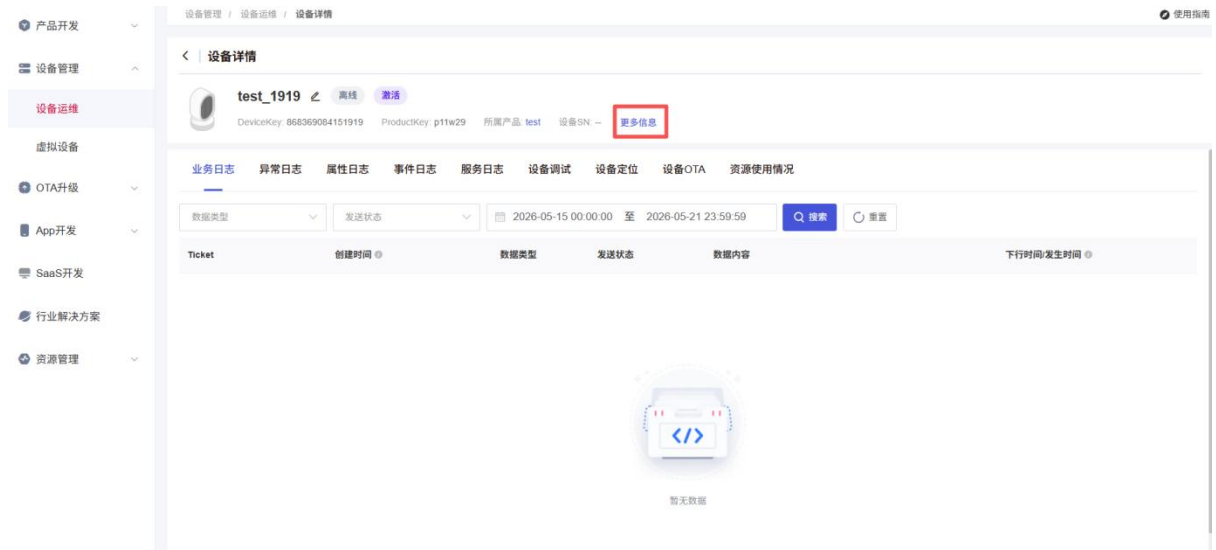
该回调需在 `setEventCb` 中通过 `"readTslServer"` 字段注册；平台下发的属性/透传数据可用 `"recvTsl"` 回调接收。

提示：更多接口与示例见官方文档 <https://iot-cloud-docs.quectelcn.com/document/quecPythonExample01>

六、APP 使用

1) 为设备创建 SN 码（可用模组 IMEI 号或 自定义字符串）

平台生成二维码后，将把 SN 列表与设备二维码图片，以压缩包的方式供用户下载，该设备二维码与模组 IMEI 完成绑定后，App 用户即可通过该设备二维码绑定控制设备。



test_1919 详细信息

基础信息

所属产品
test

创建时间
2026-05-15 15:17:43

DeviceSecret
***** 查看

最后上线时间
2026-05-15 15:17:43

认证类型
动态认证 (设备密钥)

设备SN
--

设备类型
单品设备

设备时区
--

连接协议
MQTT

硬件信息

设备标识

MNC

模组类型

BG95-M2

ICCID

ICCID

组件版本

组件版本

支持定位方式

AUTO,LBS;

协议版本

1.3.0

MCC

--



2) 下载并登录“移联万物”

微信小程序 (微信搜索对应名称)

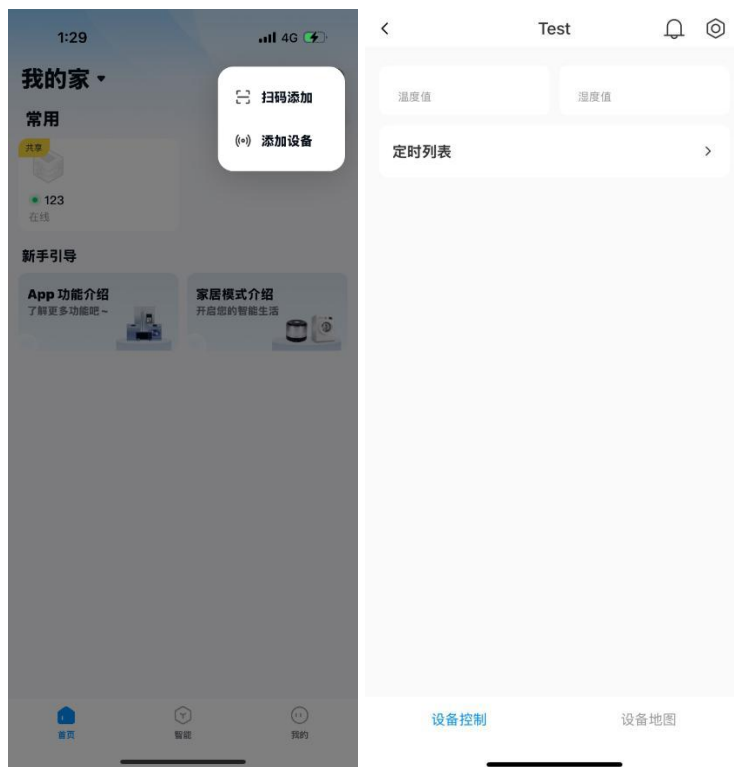
toB：移企物联

toC：移块上云

APP：移联万物（手机通过应用商店下载”移联万物”应用）

3) 扫码绑定设备

注册登录移联万物，使用扫码添加设备，即可在手机上同步出设备信息



此时设备若是上传物模型数值，则会在 APP 界面显示具体数值。